

中间代码数据结构 - 以 LLVM IR 为例

对于优化来说，一个设计良好的中间代码数据结构能够提供极大的帮助。而即便不做优化，定义中间代码相关数据结构也能为中间代码的生成提供更加结构化的操作接口。在 tolangc 中，我们参考 LLVM 的实现，提供了一个相对简单，但足以支持 tolangc 实现的 LLVM 后端，希望能够为各位同学们提供参考。

本章节只介绍了 tolangc 中 LLVM IR 的实现方式，对于 LLVM 的更多介绍见《中间代码生成 - LLVM》。

一、文法涉及的 LLVM IR 语法结构

根据 tolang 的文法和语义，我们可以得知 tolangc 所需要的 LLVM IR 语法结构有：

1. 函数块

Plain Text

```
1 define dso_local i32 @add(i32 %0, i32 %1) {}
```

2. 指令集 Instruction

指令类型	包含的 LLVM IR 指令形式
UnaryInstruction：1 个操作数	<code>+</code> <code>-</code> <code>!</code> <code>number</code> <code>load i32, i32* %1</code>
BinaryInstruction：两个操作数	<code>a + - / % b</code> <code>a > < >= <= == != b</code> <code>store i32 %0, i32* %1</code>
Instruction：其他指令	<code>alloca i32</code> <code>ret i32 %7</code> <code>call i32 @add(i32 @i1, i32 @i2)</code> <code>call i32 @get()</code> <code>call void @put(i32 %5)</code>

3. 局部变量 / 字面量

针对以上语法结构，我们设计了对应的 `Value` 类存储上下文信息。

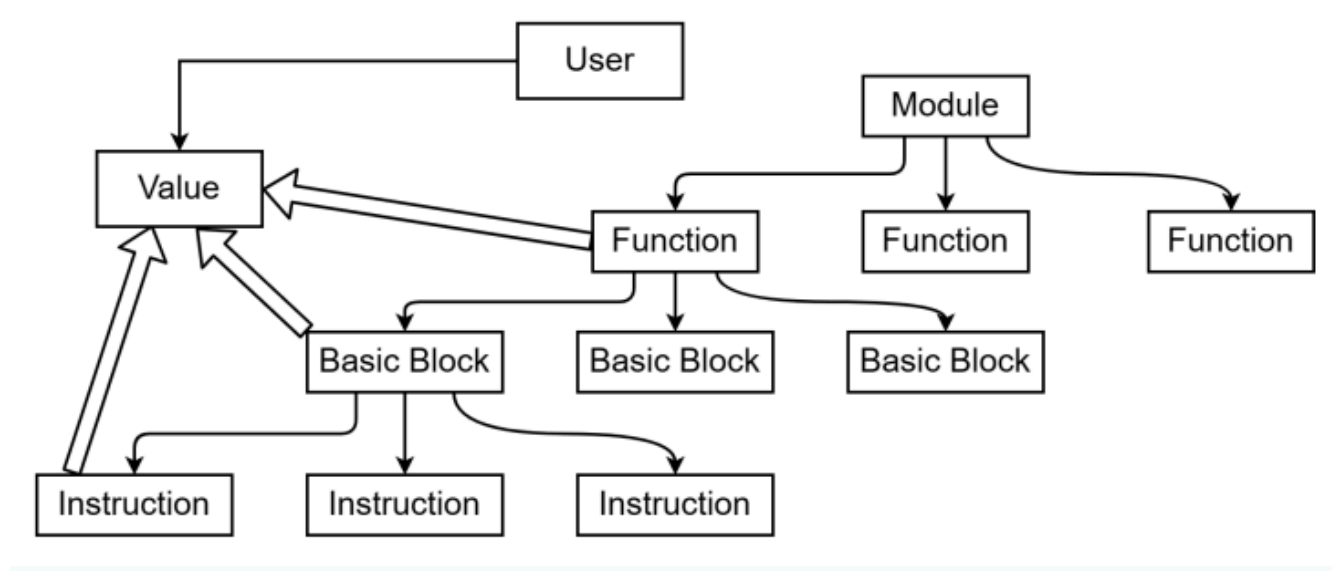
在实验编译器的实现中，也可以按需定义编译器中的 LLVM IR 数据结构。

二、架构设计

根据涉及的语法结构，我们可以把 LLVM 语法结构由粒度从高到低划分为：

1. 整个模块 `Module`
2. 函数 `Function`
3. 基本代码块 `BasicBlock` （以 label 和跳转语句为界限）
4. 指令 `Instruction`
5. 变量 / 常量 && 符号
6. 变量：参数、局部变量；常量：字面量

整体 LLVM 语法层级结构如下图所示：



先解释一下图中的 `User` 和 `Value` 类的意义。

对于模块中不同粒度的所有语法结构，我们借鉴 LLVM 官方库 (<https://github.com/llvm/llvm-project>) 的实现方法，将它们都作为 `Value` 类的子类。其中，`User` 类是 `Value` 的一个特殊的子类，是一种可以使用其他 `Value` 对象的 `Value` 类。`Function`、`BasicBlock` 和 `Instruction` 都有使用的语法结构，都是 `User` 的子类，也是 `Value` 的子类。

`User` 和 `Value` 之间的配对用 `Use` 类记录，通过 `Use` 类建立起了语法结构的上下级关系双向索引。这样所有的语法结构都可以统一成 `Value` 类，语法结构间的使用关系都可以统一成 `Use` 类。`Use` 关系是 LLVM 编译器实现的核心架构，索引关系的抽象使其能够在全局保存，可以大大提高代码优化的效率。

Plain Text

```
1 A: %add = add nsw i32 %a, %b1
2 B: %add1 = add nsw i32 %a, %b2
3 C: %add2 = add nsw i32 %add, %add13
```

对于 `Value` 和 `User` 的具体对应，可以参考以下实例代码。A 语句被看作是一个 `Instruction`，此 `Instruction` 继承自 `User`。作为 `User`，它引用了 `a` 和 `b` 两个 `Value`，又被 C 语句所引用。

在我们的设计中，`Value` 的主要属性值有：

Plain Text

```
1 class Value {
2 protected:
3     TypePtr _type;
4     std::string _name;
5
6     UseList _useList;
7     UseList _userList;
8
9 private:
10     ValueType _valueType;
11 };
```

可以看到，`Value` 类中的几个重要属性成员分别为：

- 语法类型 `ValueType _valueType`
- 值类型 `Type _type`
- 使用关系索引列表 `UseList _useList`
- 名称 `std::string _name`

接下来我们围绕 `Value` 成员讲解中端总体架构设计。

(1) 语法类型 `ValueType`

`ValueType _valueType` 中，`ValueType` 是 `Value` 的类型枚举。所有语法结构都是 `Value` 的子类，不同子类通过修改 `_valueType` 值区分：

```

1 enum class ValueType {
2     // Value
3     ArgumentTy,          // 参数
4     BasicBlockTy,        // 基本块
5
6     // Value -> Constant
7     ConstantTy,          // 常量标识符
8     ConstantDataTy,      // 字面量
9
10    // Value -> Constant -> GlobalValue
11    FunctionTy,
12    GlobalVariableTy,
13
14    // Value -> User -> Instruction
15    BinaryOperatorTy,
16    CompareInstTy,
17    BranchInstTy,
18    ReturnInstTy,
19    StoreInstTy,
20    CallInstTy,
21    InputInstTy,
22    OutputInstTy,
23    AllocaInstTy,
24    LoadInstTy,
25    UnaryOperatorTy,
26 };

```

(2) 值类型 Type

`TypePtr _type` 记录的是返回值类型，与 `ValueType` 不属于一个系统：

- `ValueType` 用于区分我们自行实现的 LLVM 架构中定义的不同数据类型。
- `Type` 记录的是 LLVM 本身语法结构中的数据类型，可以理解为一个指令的返回值类型。

比如对于指令对象：`%add = add i32 %a, %b`，返回值类型 `Type` 为 `%add` 的类型 `i32`，指令类型 `ValueType` 为二元操作数指令类型 `BinaryOperatorTy`。

`Type` 可以区分的数据类型包括：

Plain Text

```
1 enum TyID {
2     // Primitive types
3     VoidTyID,      // 空返回值
4     LabelTyID,     // 标签类型
5
6     // Derived types
7     IntegerTyID,   // 整数类型
8     FloatTyID,     // 浮点数类型
9     FunctionTyID,  // 函数类型
10    PointerTyID    // 指针类型
11 };
```

(3) 索引 Use & User

`User` 是 `Value` 的子类。通过 `User` 和 `Value` 的配对，索引 `Value` 间的使用和被使用关系。

目前 `Use` 关系只用于记录指令和操作数的关系。每条指令建立的同时，我们将操作数作为 `Use`，指令作为 `User`，创建 `Use` 对象，将 `Use` 对象同时保存在全局记录和 `User` 中的 `uselist` 属性中。指令和每个操作数之间都是一个 `Use` 关系。每个 `User` 的 `Use` 关系数量不定（一条指令中的操作数数量不定，参数调用中参数个数不限）。

我们使用 `LlvmContext` 语法结构记录全局的 `Use` 关系。每个编译单元 `Module` 中只有一个 `LlvmContext`：

Plain Text

```
1 class LlvmContext {
2     std::vector<FunctionTypePtr> _functionTypes;
3     std::vector<PointerTypePtr> _pointerTypes;
4
5     std::vector<ValuePtr> _values;
6     std::vector<UsePtr> _uses;
7 };
```

`LlvmContext` 存储的是对象的指针（引用）。

记录对象之间的索引关系为之后的代码优化提供便利，让我们能查找到某个变量的所有使用地点，更方便地进行变量替换。设计的代码优化方法有活跃变量分析、公共子表达式删除等。

(4) 名称记录

`std::string _name` 中记录语法结构的名称。名称记录只在全局变量（`GlobalVariable`、`Function`）中有意义。

本文法编译不涉及全局变量。但代码库中也完成了全局变量的指令实现，可以作为其他编译器实现的参考。

局部变量的名称在 LLVM 中没有意义，因为在翻译过后需要用数字对虚拟寄存器重新命名——按出现顺序为虚拟寄存器编号（LLVM 的虚拟寄存器相关知识见课程教程）。`Value` 对象中通过 `Use` 保存域内各个成员结构的指针索引，成员间的使用关系中无需语法对象名称信息，即整个翻译过程与对象名称无关，与虚拟寄存器编号也无关。因此，分配虚拟寄存器编号不用在翻译过程中实现，可以推迟到代码打印的部分。

在我们的实现中，`SlotTracker` 类作为一个工具类，在中间代码打印时为一个函数域分配虚拟寄存器，记录各语句对应的虚拟寄存器编号。

Plain Text

```
1 class SlotTracker final {
2     std::unordered_map<ValuePtr, int> _slot;
3 };
```

在中间代码打印前使用 `SlotTracker::Trace` 方法，按出现顺序建立 `ValuePtr` 和编号值之间的索引；打印时通过 `SlotTracker::Slot` 方法获取对应寄存器的编号。

三、LLVM 文件结构说明

最后，我们给出 tolangc 中 LLVM 模块的文件结构说明。

Plain Text

```
1 +---asm
2 |      AsmWriter.h      // 存储中间代码字符串
3 |      AsmPrinter.h     // 打印 writer 中存储的结构化字符串
4 \---ir
5      |      IrForward.h  // 各个语法结构前向声明（防止交叉引用报错）：定义指针类型
6      |      Llmv.h       // 罗列 LLVM 翻译过程中所需的头文件集合
7      |      Module.h     // LLVM 的一个编译单元，按文法定义存储：函数列表、main
      函数、上下文语义记录 LlmvContext
8      |      LlmvContext.h // LLVM 编译单元 Module 的语义记录，保存类型字典、Us
      e 关系（一个 Module 对应一个 LlmvContext）
9      |      SlotTracker.h // 为虚拟寄存器编号和输出的工具类
10     |      Type.h        // 定义 LLVM 的对象类型和类型子类，类型包括：Void、Lab
      el、Integer、Function、Pointer
11     \---value           // LLVM 翻译中自行定义的语法单元
12         |      Value.h      // 所有语法单元的父亲，主要记录语法单元类型
      和 Use 关系
13         |      Use.h        // 一个 Use 对象记录一个 User 和 Value 对
14         |      User.h       // Value 的子类
15         |      ArgRegTy.h   // 参数 Value
16         |      BasicBlock.h // 代码块 Value
17         |      Constant.h   // 常量 Value
18         |      ConstantData.h // 字面量 Value -> Constant
19         |      GlobalValue.h // 全局变量 Value -> Constant
20         |      Function.h    // 函数 Value -> Constant -> GlobalValu
      e
21     \---inst            // 指令 Value -> User
22         Instruction.h       // 指令基本类
23         InstructionTypes.h  // 定义存在类型区分的指令类：UnaryIn
      struction -> UnaryOperator, BinaryInstruction -> BinaryOperator & Compa
      reInstruction
24         Instructions.h      // 定义 LLVM 固有指令：AllocaIns
      t、LoadInst、StoreInst、BranchInst、JumpInst、ReturnInst、CallInst
25         ExtendedInstructions.h // 定义外部库指令：InputInst、Outp
      utInst
```